

Kimlik ve Eriřim Yönetimi

[Kaynak](#)

İÇERİK

1. IAM (Identity and Access Management) Temelleri: Kimlik, Yetkilendirme ve Hesap Verilebilirlik
2. Bilgi Güvenliđi Mimarisi: IAAA Modeli (Identification, Authentication, Authorisation, Accountability)
3. Identification (Kimlik Tanımlama)
4. Authentication (Kimlik Doğrulama) ve MFA Mimarisi
5. Authorisation ve Access Control Mimarisi
6. IAM ve Güvenlik Mimarisinde Accountability (Hesap Verilebilirlik) ve Merkezi Loglama (Logging)
7. IAM ve IdM Mimarisi: Kimlik ve Eriřim Yönetimi Dinamikleri
8. Kimlik Doğrulama Mekanizmalarına Yönelik Saldırıları (Attacks Against Authentication)
9. Eriřim Kontrol Modelleri (Access Control Models)
10. IAM Mimarisi: Single Sign-On (SSO) ve Merkezi Kimlik Doğrulama

IAM (Identity and Access Management) Temelleri: Kimlik, Yetkilendirme ve Hesap Verilebilirlik

Herhangi bir sistemin güvenlik mimarisi, kullanıcıların sistemle olan etkileřimini yöneten temel soruların kusursuz bir řekilde yanıtlanmasına dayanır. Bir saldırganın meřru bir kullanıcı gibi davranmasını engellemek veya bir kullanıcının sistem içindeki eylemlerinden sorumlu tutulabilmesini sağlamak, birbirine bađlı bir dizi güvenlik sürecinin doğru işletilmesiyle mümkündür.



Bu bağlamda sistem güvenliği; kimlik belirleme, doğrulama, yetkilendirme ve izleme aşamalarından oluşan doğrusal bir güvenlik boru hattı (security pipeline) olarak inşa edilir.

Benzersiz Kullanıcı Tanımlaması: Identification

Sistemdeki farklı kullanıcıları birbirine karışmadan, benzersiz bir şekilde ayırt etme sürecine **Identification** (Kimlik Tanımlama) adı verilir. Kullanıcının sisteme "Ben kimim?" sorusunun cevabını sunduğu ilk aşamadır.

Identification, tek başına bir güvenlik önlemi değildir; sadece bir beyandır. Kullanıcı adı (username), e-posta adresi veya bir çalışan numarası bu beyanı temsil eder. Teknik olarak, sistemin veri tabanında (örneğin Linux sistemlerde `/etc/passwd` dosyasında veya Windows'ta SAM veritabanında) bu kimliğin bir karşılığı olup olmadığı kontrol edilir.

```
cat /etc/passwd | grep root  
root:x:0:0:root:/root:/bin/bash
```

Yukarıdaki çıktı, bir Linux sisteminde 'root' kullanıcısının kimlik tanımlamasını (Identification) gösterir. Sistem, 'root' ismini gördüğünde bunun 0 numaralı UID'ye (User ID) sahip kullanıcı olduğunu anlar. Ancak bu aşama, o an klavye başındaki kişinin gerçekten 'root' olup olmadığını kanıtlamaz.

Kimlik Beyanının Kanıtlanması: Authentication

Kullanıcının sisteme sunduğu kimliğin gerçekten kendisine ait olduğunu teknik olarak ispatlaması sürecine **Authentication** (Kimlik Doğrulama) denir. "Ben X kullanıcısıyım" beyanının (Identification) ardından sistemin "Bunu bana kanıtla" dediği aşamadır.

ÖNEMLİ: Sektörde ve sınavlarda en sık yapılan hata, Identification ile Authentication kavramlarını birbirine karıştırmaktır. Bir e-posta adresi girmek Identification, o e-posta adresine ait parolayı girmek Authentication'dır.

Saldırganların meşru kullanıcıları taklit etmesini (spoofing/impersonation) engellemek için sadece parola tabanlı doğrulama genellikle yetersiz kalır. Bu noktada savunma derinliğini (defense in depth) artırmak için iki kritik konsept devreye girer:

- Strong Passwords** (Güçlü Parolalar): Brute-force (kaba kuvvet) ve dictionary (sözlük) saldırılarına karşı entropisi yüksek, karmaşık karakter dizileridir.
- Multi-Factor Authentication - MFA** (Çok Faktörlü Kimlik Doğrulama): Doğrulama sürecini tek bir zafiyet noktası (Single Point of Failure) olmaktan çıkarır. Kullanıcının bildiği bir şey (parola), sahip olduğu bir şey (donanımsal token, telefon uygulaması) veya fiziksel bir özelliği (biyometrik veri) gibi farklı faktörlerin en az ikisinin aynı anda başarılı bir şekilde sunulmasını zorunlu kılar.

Eriřim Kararlarının Uygulanması: Authorisation ve Access Control

Kullanıcı kimliğini başarıyla kanıtladıktan (Authentication) sonra, sistemin bu kullanıcıya hangi dosyalara, dizinlere veya servislere erişim izni vereceğine karar vermesi sürecine **Authorisation** (Yetkilendirme) denir.

Bir kullanıcının sisteme giriş yapabilmesi, sistemdeki her şeyi okuyabileceği veya değiştirebileceği anlamına gelmez. "Kullanıcı sisteme girdi, peki şimdi ne yapabilir?" sorusunun cevabı burada yatar. Bu kararların sistem seviyesinde teknik olarak dayatılması ve sınırlandırılması mekanizmasına ise **Access Control** (Eriřim Kontrolü) modelleri denir.

```
ls -la /etc/shadow
-rw-r----- 1 root shadow 1234 May 12 10:00 /etc/shadow
```

Bu çıktı, Linux sistemlerinde parolaların hash (özet) değerlerini tutan kritik `/etc/shadow` dosyasına ait erişim kontrolü (Access Control) yapısını gösterir. Dosya üzerindeki `-rw-r-----` izinleri, sistemin **Authorisation** mantığını açıkça yansıtır: Yalnızca dosyanın sahibi olan **root** kullanıcısı okuma ve yazma (rw) yetkisine sahiptir. Standart bir kullanıcı, sisteme başarıyla **Authentication** yapmış olsa bile, **Authorisation** kuralları gereği bu dosyayı okuyamaz.

Sistem İçi İzlenebilirlik: Logging ve Auditing

Kullanıcı doğrulandı ve yetkilendirildi. Peki kullanıcının sistem içinde yaptığı eylemlerin meşru olup olmadığını, bir veri ihlali durumunda failin kim olduğunu nasıl tespit edeceğiz? Bir kullanıcının eylemlerinden sorumlu tutulabilmesi konseptine **Accountability** (Hesap Verilebilirlik) denir.

Bu sorumluluğun teknik altyapısı iki aşamadan oluşur:

1. **Logging** (Kayıt Tutma): Sistemde gerçekleşen oturum açma, dosya okuma, komut çalıştırma gibi eylemlerin zaman damgasıyla (timestamp) birlikte kaydedilmesidir.
2. **Auditing** (Denetleme): Tutulan bu logların, güvenlik politikalarına uyumluluk, anomali tespiti veya olay müdahalesi (Incident Response) kapsamında proaktif veya reaktif olarak analiz edilmesidir.

```
May 12 10:15:32 server sshd[12345]: Failed password for invalid user admin
from 192.168.1.50 port 54321 ssh2
May 12 10:15:35 server sshd[12345]: Accepted publickey for root from
10.0.0.5 port 2222 ssh2
```

Tipik bir `/var/log/auth.log` çıktısı. Birinci satırda, var olmayan bir kimlikle (Identification hatası) yapılan başarısız bir **Authentication** girişimi loglanmıştır (Logging). İkinci satırda ise MFA / Public Key altyapısı kullanılarak başarılı bir sistem girişi kaydedilmiştir. Olası bir siber saldırı durumunda, sistem yöneticisi (veya Blue Team analisti) bu logları denetleyerek

*(**Auditing**), saldırganın hangi IP adresinden (192.168.1.50) geldiğini ve hangi kullanıcı hesaplarını hedef aldığını tespit ederek ilgili sistemdeki güvenlik mimarisini yeniden yapılandırmak için gereken adımları atacaktır.*

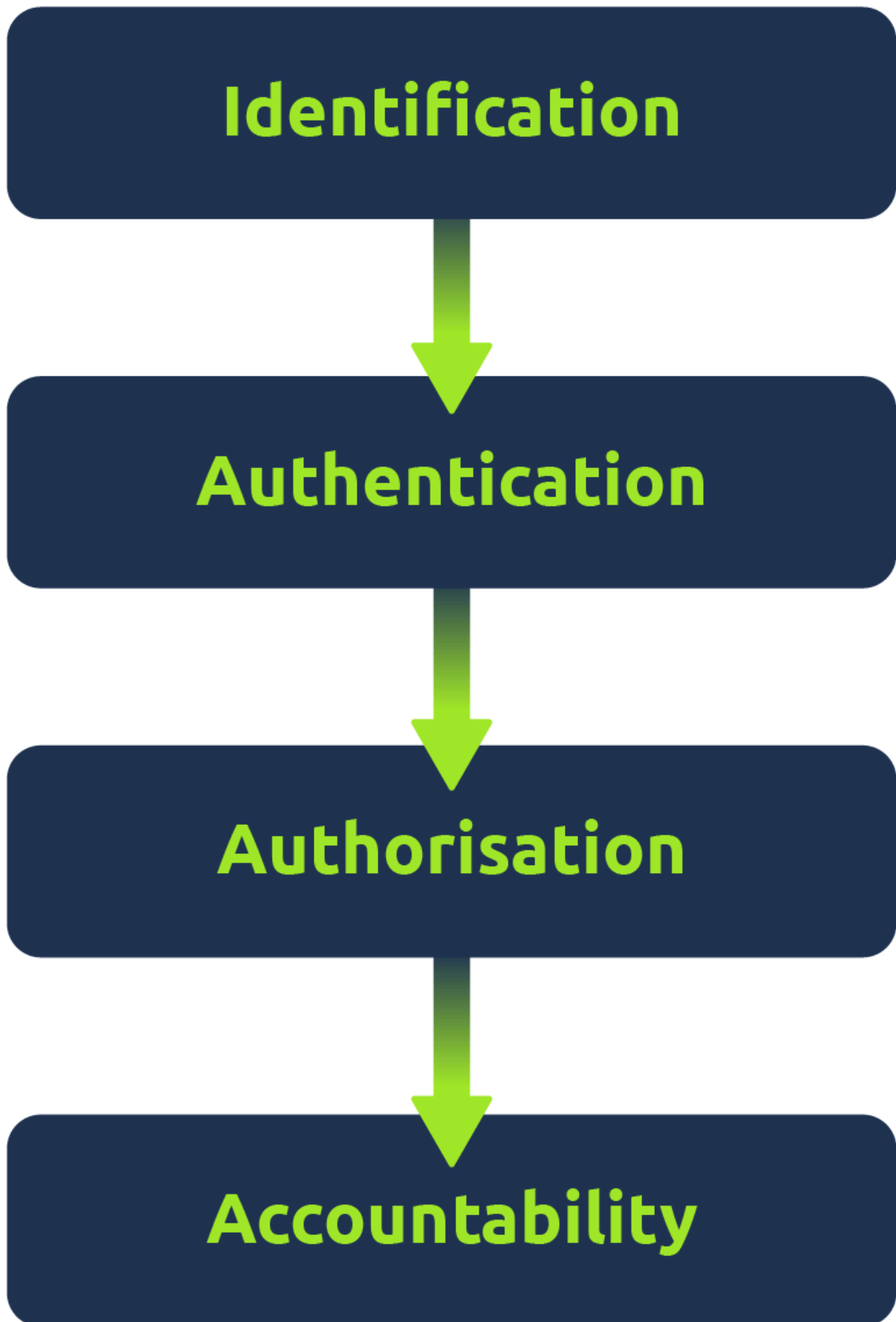
DİKKAT: Güvenlik mimarisinde bu kavramlar birbirinin alternatifi değil, birbirini tamamlayan zincirlerdir. Başarılı bir **Identification** olmadan **Authentication** yapılamaz, **Authentication** olmadan **Authorisation** tahsis edilemez ve doğru yapılandırılmış bir **Logging** mekanizması olmadan tüm bu sistemin **Accountability** (Hesap Verilebilirlik) özelliği sağlanamaz. Aylar sonra bu konuya dönüldüğünde hatırlanması gereken en kritik mantık zinciri budur.

Bilgi Güvenliği Mimarisi: IAAA Modeli (Identification, Authentication, Authorisation, Accountability)

Bilgi güvenliği mimarisinde hassas verilerin ve sistem kaynaklarının gizliliğini (Confidentiality), bütünlüğünü (Integrity) ve erişilebilirliğini (Availability) sağlamak, sisteme kimin, nasıl ve hangi sınırlar dahilinde eriştiğinin kesin bir şekilde kontrol edilmesine bağlıdır. Bu yapısal denetim mekanizmasının temelini, dört ana sütundan oluşan **IAAA Modeli** oluşturur.

Bu model, yalnızca teorik bir çerçeve değil; bir kullanıcının sisteme ilk temasından, gerçekleştirdiği eylemlerin adli analizine (forensics) kadar uzanan teknik yaşam döngüsünün

eksiksiz bir haritasıdır.



1. Identification (Kimlik Tanımlama)

Sisteme erişim sürecinin "Sen kimsin?" sorusuyla başlayan ilk adımıdır. **Identification**, kullanıcının belirli bir kimlik üzerinde hak iddia etmesi (claim) işlemidir. Bu aşama, kendi başına bir güvenlik doğrulaması sağlamaz; yalnızca kullanıcının sistem veri tabanında aranacak olan etiketini sunmasıdır.

Bu kimlik beyanı, sistemin ilgili ortamında (environment) tamamen benzersiz (unique) olan bir tanımlayıcı üzerinden yapılır. E-posta adresi, kullanıcı adı (username) veya bir çalışan kimlik numarası (ID number) bu amaca hizmet eder.

Neden e-posta adresleri daha çok tercih edilir?

Geniş çaplı web uygulamalarında veya kurumsal ağlarda, kullanıcı adlarının çakışma (collision) ihtimali yüksektir (örneğin birden fazla "ahmet.yilmaz" olması). E-posta adresleri ise doğası gereği global ölçekte benzersiz oldukları için modern kimlik tanımlama süreçlerinde birincil **Unique Identifier** olarak kullanılır.

2. Authentication (Kimlik Doğrulama)

Kullanıcının **Identification** aşamasında iddia ettiği kimliğin gerçekten kendisine ait olduğunu teknik olarak ispatladığı aşamadır. Sistem bu noktada kullanıcıya "Bunu bana kanıtla" der. Başarılı ve geçerli bir kimlik tanımlaması olmadan kimlik doğrulama sürecine geçilemez.

Geleneksel sistemlerde bu doğrulama, kullanıcının sisteme doğru parolayı (password) sunmasıyla gerçekleşir. Ancak, yalnızca parola tabanlı doğrulamalar; Brute-Force (Kaba Kuvvet) veya Credential Stuffing (Kimlik Bilgisi Doldurma) gibi saldırı vektörlerine karşı kritik zafiyetler barındırır. Bu zayıflığı gidermek ve doğrulama mekanizmasını güçlendirmek amacıyla, kullanıcının e-posta adresine/telefonuna gönderilen doğrulama kodları veya donanımsal token'lar gibi ek güvenlik katmanları içeren **MFA (Multi-Factor Authentication - Çok Faktörlü Kimlik Doğrulama)** yapıları sektör standardı haline gelmiştir.

3. Authorisation (Yetkilendirme)

Kullanıcı kimliğini başarıyla doğruladıktan sonra, sistemdeki tüm kaynaklara sınırsız erişim hakkı elde edemez. **Authorisation**, doğrulanmış kullanıcının sistem içerisinde hangi dosyalara, dizinlere veya servislere erişmesine (access) izin verileceğini belirleyen mekanizmadır.

Bu süreç, En Az Ayrıcalık Prensibi (Principle of Least Privilege - PoLP) temel alınarak inşa edilir. Kullanıcılara yalnızca iş tanımlarını (job function) yerine getirebilecekleri veya sahip oldukları güvenlik izin seviyesinin (clearance level) elverdiği minimum yetkiler, spesifik roller (Roles) ve izinler (Permissions) aracılığıyla atanır.

Teknik Derinlik: Yetkilendirme adımının bu kadar sıkı tutulmasının temel sebebi, olası bir hesap ele geçirme (Account Takeover) durumunda saldırganın sistem içindeki hareket alanını kısıtlamaktır. Eğer sıradan bir çalışanın hesabı ele geçirilirse, doğru yapılandırılmış bir Authorisation mekanizması sayesinde saldırganın kritik verilere erişimi (Data Breach) veya sistem yetkilerini yükseltmesi (Privilege Escalation) engellenmiş olur.

4. Accountability (Hesap Verilebilirlik)

Güvenlik mimarisinin en kritik son halkasıdır. Kullanıcı doğrulandı, yetkilendirildi ve sisteme girdi. Peki, sistem içinde ne yaptığını nasıl bileceğiz? **Accountability**, kullanıcının sistem üzerinde gerçekleştirdiği tüm aktivitelerin izlenmesi (tracking) ve eylemlerinden sorumlu tutulabilmesi sürecidir.

Bu hesap verilebilirlik durumu, tüm kullanıcı eylemlerinin bir zaman damgasıyla (timestamp) birlikte kaydedilmesi (Logging) ve merkezi bir sunucuda güvenle saklanmasıyla elde edilir.

```
# Örnek bir Linux /var/log/auth.log Çıktısı Analizi
May 15 14:22:10 srv-web-01 sshd[10432]: Accepted password for developer_john
from 192.168.1.55 port 49152 ssh2
May 15 14:25:01 srv-web-01 sudo[10455]: developer_john : TTY=pts/0 ;
PWD=/home/developer_john ; USER=root ; COMMAND=/bin/cat /etc/shadow
```

Yukarıdaki log kaydı, IAAA modelinin pratikteki en net göstergesidir. İlk satırda "developer_john" **Identification** sunmuş ve parolası kabul edilerek **Authentication** (Accepted password) sağlanmıştır. Ancak ikinci satırda, `sudo` kullanarak kritik `/etc/shadow` dosyasını okumaya çalışmıştır. Eğer bu kullanıcının **Authorisation** seviyesi bu dosyayı okumaya yetmiyorsa işlem reddedilecektir. Ve en önemlisi, kullanıcının bu şüpheli eylemi, IP adresi (192.168.1.55) ve tam zamanıyla (May 15 14:25:01) loglanarak **Accountability** (Hesap Verilebilirlik) sağlanmıştır.

ÖNEMLİ: Olası bir güvenlik ihlalinde (Security Incident), Incident Response (Olay Müdahale) ekipleri bu logları kullanarak problemin kaynağını (Source of the problem) tespit eder, saldırının yatay/dikey nasıl ilerlediğini haritalandırır ve yasal olarak faili eylemleriyle eşleştirir (Non-repudiation - İnkâr edememezlik).

Sonuç ve Mimari Bütünlük

IAAA modeli, yetkisiz erişimleri (unauthorised access), veri sızıntılarını ve diğer siber güvenlik olaylarını önleyen bir savunma hattıdır. Kurumlar, bu dört adımı (Identification → Authentication → Authorisation → Accountability) zincirleme bir reaksiyon olarak, eksiksiz bir şekilde entegre (best practices) ettiklerinde; sistemlerini hem içeriden gelebilecek tehlikelere (Internal Threats) hem de dışarıdan gelebilecek saldırılara (External Threats) karşı proaktif olarak korumuş olurlar.

Identification (Kimlik Tanımlama)

Siber güvenlik mimarisinde, bir sisteme erişim sürecinin ilk ve en temel adımı kullanıcıların, süreçlerin (process) veya diğer sistemlerin kendilerini tanıtmalarıdır. Bu süreç teknik terminolojide **Identification** (Kimlik Tanımlama) olarak adlandırılır.

Identification (Kimlik Tanımlama) Nedir?

Identification, bir varlığın (kullanıcı, cihaz veya arka plan servisi) sisteme belirli bir kimlik üzerinde "hak iddia etmesi" (claim) işlemidir. Sistem ile etkileşime giren tarafın, "Ben kimim?" sorusuna verdiği yanıtıdır.

Bu kavramın doğasını anlamak için günlük hayattan, düşük güvenli (low-security) bir senaryoyu ele alalım: Sosyal bir etkinliğe katıldığınızda birisi size adınızı sorduğunda gerçek adınızı söyleyebilir veya kendinizi favori filminizden bir karakterin adıyla, örneğin "Thomas Anderson" olarak tanıtabilirsiniz. Ortam bir yüksek güvenlik alanı olmadığı ve sadece sosyal bir etkileşim barındırdığı için, karşınızdaki kişi sizden kimliğinizi (ID) ispatlamanızı istemeyecektir. Sadece beyanınıza (claim) güvenir.

Ancak bilişim sistemlerinde ve kurumsal ağlarda bu beyan, erişim kontrolü (Access Control) mekanizmasının başlatıcı tetikleyicisidir.

Sistemlerde Identification Pratikleri ve Formatları

Bilişim sistemleri, **Identification** sürecini yönetmek için çeşitli formatlar kullanır. Kullanıcının kimlik beyanı, kurumun belirlediği isimlendirme politikalarına (Naming Convention Policy) veya platformun altyapısına göre değişiklik gösterir:



1. **Username (Kullanıcı Adı):** Bir kullanıcının sistemdeki mantıksal karşılığıdır. Örneğin, Thomas Anderson isimli bir kullanıcı için kurumun politikasına göre **tanderson**, **thomasa**, **thomas01**, **ta001** veya sadece **neo** gibi bir kullanıcı adı belirlenebilir.
2. **Unique Identifiers (Benzersiz Tanımlayıcılar):** Kimlik tanımlamasının sistem içerisinde teknik olarak geçerli olabilmesi için, beyan edilen değer o ortamda (environment) tamamen benzersiz olması gerekir. Bu nedenle T.C. Kimlik Numarası, Öğrenci Numarası, Pasaport Numarası veya Cep Telefonu Numarası gibi tekil değerler sıklıkla **Identification** parametresi olarak kullanılır.

Neden modern sistemler Username yerine Email kullanıyor? Birçok web uygulaması ve modern platform, kayıt aşamasında özel bir kullanıcı adı (username) oluşturmak yerine doğrudan bir e-posta adresi talep eder. Bunun arkasındaki teknik mantık oldukça basittir: E-posta adresleri doğası gereği global ölçekte benzersizdir (guaranteed to be unique). Bu yaklaşım, sistem veri tabanında isim çakışmalarını (collision) sıfıra indirir ve kullanıcının benzersiz bir kullanıcı adı bulma/hatırlama yükünü (user friction) ortadan kaldırarak verimliliği artırır.

```
-- Bir sistemin arka planında Identification beyanını arayan örnek bir SQL sorgusu
SELECT user_id, email, status FROM users WHERE email =
'thomas.anderson@matrix.com';
```

Yukarıdaki örnekte sistem, kullanıcının beyan ettiği e-posta adresini (Identification) veri tabanında sorgulamaktadır. Eğer bu kayıt mevcutsa, sistem kullanıcının kimlik beyanını tanır ve bir sonraki aşama olan ispat aşamasına geçer.

Identification'dan Authentication'a (Kimlik Doğrulama) Geçiş ve Zorunluluğu

Bir bilgisayar sisteminin dışında, örneğin yeni kayıt olduğunuz bir spor salonuna gittiğinizi düşünün. Resepsiyonist isminizi sorduğunda (Identification talebi), yine favori filminizden bir isim verebilirsiniz. Ancak bu kez resepsiyonist sizden kimlik kartınızı (ID card) göstermenizi isteyecektir.

Neden? Çünkü sistem (spor salonu yönetimi), aidat ödeyen meşru bir abonenin ismini kullanan rastgele birinin (impersonation/spoofing) içeri girmesini engellemek zorundadır. İşte beyan edilen kimliğin gerçekten o kişiye ait olduğunun kanıtlanması sürecine **Authentication** (Kimlik Doğrulama) denir.

ÖNEMLİ: Identification ile Authentication siber güvenlikteki en çok karıştırılan iki farklı konsepttir. Bir e-posta adresi veya kullanıcı adı girmek **Identification** (Kimlik Beyanı), o adrese ait parolayı girmek ise **Authentication** (Kimlik Doğrulamasıdır).

Eğer sistemler güçlü bir **Authentication** mekanizması talep etmeden sadece **Identification** ile çalışsaydı, ortaya çıkacak hasarların (severe damage) boyutu felaket düzeyinde olurdu:

- Bankacılık sistemlerinde sahte bir kimlik beyanıyla adınıza kredi çekilebilir (Financial Fraud).
- Siber dünyada, sadece e-posta adresinizi (Identification) bilen herhangi bir saldırgan, parolanız (Authentication) sorulmadığı için tüm özel yazışmalarınıza erişebilir (Data Breach).

Otel rezervasyon ağlarından uçuş sistemlerine, bankacılık altyapılarından kurumsal Active Directory ortamlarına kadar hiçbir bilgi sistemi, **Identification** aşamasını **Authentication** ile

desteklemeden güvenli ve işlevsel bir şekilde çalışamaz. Her yetkilendirme (Authorisation), öncelikle doğrulanmış bir kimliğe dayanmak zorundadır.

Authentication (Kimlik Doğrulama) ve MFA Mimarisi

Bir bilgi sisteminin veya ağın güvenliği, kaynaklara erişmeye çalışan varlıkların kimliklerinin mutlak bir kesinlikle kanıtlanmasına dayanır. Erişim kontrolü (Access Control) mimarisinin kalbinde yer alan **Authentication** (Kimlik Doğrulama) süreci, sistem güvenliğinin en kritik bariyeridir.

Bu süreci teknik bir derinlikle kavrayabilmek için öncelikle **Identification** (Kimlik Tanımlama) ile arasındaki kesin çizgiyi çizmek gerekir.

Identification vs. Authentication: Teknik Farklar

Identification, bir kullanıcının, sistemin veya işlemin (process) ilgili ortamda belirli ve benzersiz (unique) bir kimlik üzerinde hak iddia etmesidir (claim). Sistemde bir kullanıcı adı veya e-posta adresi girmek sadece bir beyandır.

Authentication ise, iddia edilen bu kimliğin kriptografik, mantıksal veya fiziksel kanıtlarla doğrulanması (proving) işlemidir.

ÖNEMLİ: Sadece **Identification** ile çalışan bir sistem, siber güvenlikte "Sıfır Güven" (Zero Trust) mimarisinin tam zıttıdır. Parola veya ek doğrulama istemeyen bir sistemde, saldırganlar sadece geçerli bir e-posta adresi bilerek (veya tahmin ederek) tüm sisteme erişebilir ve yetki yükseltme (Privilege Escalation) veya veri sızıntısı (Data Breach) gibi kritik olaylara neden olabilirler.

Sistemlerin, beyan edilen kimliği doğrulamak için kullandığı teknik altyapılar üç ana kategoriye ayrılır.

1. Something You Know (Bildiğiniz Bir Şey)

Kimlik doğrulamanın en eski ve en yaygın formudur. Kullanıcının zihinsel olarak ezberlediği veya gizli tuttuğu bir bilgiye dayanır.



- **Passwords (Parolalar):** 4SNoPawKkdFiCdm gibi karmaşık karakter dizileridir.
- **Passphrases (Parola Cümleleri):** "Baggage Protection Dissatisfy Barrel 8" gibi, Brute-Force (Kaba Kuvvet) saldırılarına karşı entropisi (karmaşıklığı ve uzunluğu) daha yüksek olan kelime gruplarıdır.
- **PIN (Personal Identification Number):** Genellikle mobil cihazlarda veya donanım kilitlerinde kullanılan numerik dizilerdir.
- **Pattern (Geometrik Desenler):** Akıllı telefonlarda noktaları birleştirerek çizilen desenler de teknik olarak hafızaya dayalıdır ve arka planda bir PIN gibi işlenir.

Kod snippet'i

```
[AUTH_EVENT] User: tanderson | IP: 192.168.1.15 | Method: Password | Status: SUCCESS
```

Yukarıdaki örnek log kaydında, sistemin 'tanderson' kimlik beyanını (Identification) kabul ettikten sonra, kullanıcının sunduğu gizli bilgi (Password) üzerinden başarılı bir **Authentication** gerçekleştirdiği görülmektedir. Ancak bu yöntem tek başına yetersizdir; parolalar Phishing (Oltalama) veya Keylogger saldırılarıyla ele geçirilebilir.

2. Something You Have (Sahip Olduğunuz Bir Şey)

Bu faktör, kullanıcının fiziksel olarak elinde bulundurduğu bir nesneye veya cihaza dayanır. Mantık şudur: Saldırgan parolanızı uzaktan çalabilir, ancak fiziksel cihazınızı dünyanın öbür ucundan çalamaz.

- **Mobil Cihaz / SIM Kart Sahipliği:** Bir uygulamaya kayıt olurken veya giriş yaparken SMS ile gelen veya arama yoluyla iletilen tek kullanımlık kodlar (OTP - One Time

Password). Bu işlem, arka planda o numaraya ait SIM kartın (veya eSIM'in) fiziksel olarak kullanıcının kontrolünde olduğunu kanıtlar.

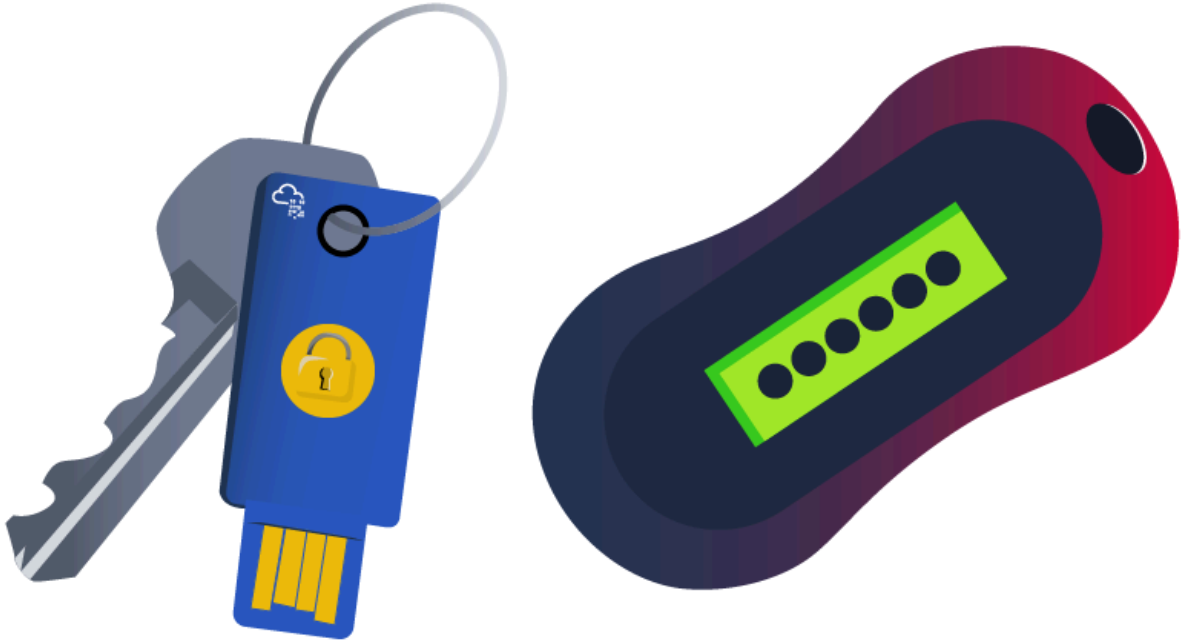
- **Hardware Security Keys (Donanımsal Güvenlik Anahtarları):** Kriptografik anahtarlar barındıran, USB, USB-C veya NFC (Yakın Alan İletişimi) üzerinden çalışan donanımlardır. **Yubico**, **Titan Security Key**, **Nitrokey** ve **Thetis** en bilinen örnekleridir. Cihaza fiziksel temas gerektirdiği için ortadaki adam (MitM) ve Phishing saldırılarına karşı en yüksek direnci sağlarlar.

3. Something You Are (Olduğunuz Bir Şey - Biyometri)

Kullanıcının değişmez fiziksel veya davranışsal özelliklerini kullanan, modern sistemlerin en güçlü doğrulamalarından biridir.

- Parmak izi okuyucuları (Fingerprint readers)
- Yüz tanıma sistemleri (Facial recognition)
- Retina tarayıcıları
- Ses tanıma (Voice recognition)

Biyometrik doğrulama sistemleri donanımsal olarak daha erişilebilir hale geldikçe son kullanıcı cihazlarında standartlaşmıştır. Ancak biyometrik sensörlerin okuma hatalarına (False Rejection Rate) karşı, sistemler her zaman bir "Bildığınız Bir Şey" (örneğin yedek PIN) faktörünü kurtarma (fallback) mekanizması olarak bekletir.



DİKKAT: Daha az kullanılan ancak gelişmiş ağlarda uygulanan iki faktör daha vardır:

Somewhere you are (Coğrafi/Mantıksal konum - örn: VPN'e sadece şirket IP bloğundan bağlanabilme) ve **Something you do** (Davranışsal analiz - örn: kullanıcının klavye yazma hızı ve mouse hareket profili).

Savunma Derinliği: Multi-Factor Authentication (MFA) ve 2FA

Siber güvenlikte hiçbir savunma katmanı tek başına aşılamaz değildir. Bu nedenle, tek bir güvenlik zaafiyetinin (Single Point of Failure) tüm sistemi tehlikeye atmasını önlemek için **Multi-Factor Authentication (MFA)** mimarisi kullanılır.

MFA, yukarıda bahsedilen (Bildiğiniz, Sahip Olduğunuz, Olduğunuz) faktörlerden **iki veya daha fazlasının aynı anda** başarılı bir şekilde sunulmasını zorunlu kılar. Sadece iki faktör kullanıldığında buna spesifik olarak **2FA (Two-Factor Authentication)** denir.

Gerçek Hayat ve Sistem Mimarisi Bağlantısı:

- **ATM Cihazları:** Sektördeki en eski 2FA örneklerindendir. Banka kartını takmanız (**Something you have**) ve PIN kodunu girmeniz (**Something you know**) gerekir. Saldırgan kartınızı kopyalasa bile (Skimming), PIN kodunu bilmeden işlem yapamaz.
- **Fiziksel Kasalar:** Modern ev kasalarında sadece fiziksel bir anahtar (**Something you have**) yetmez, aynı zamanda bir dijital PIN (**Something you know**) girilmesi istenir.

```
{
  "auth_request": {
    "user_id": "tanderson",
    "factors_required": ["password", "hardware_token"],
    "step_1_status": "VALID_PASSWORD",
    "step_2_status": "PENDING_U2F_TOKEN"
  }
}
```

Bu JSON mimari örneği, arkasında sağlam bir MFA yapısı olan uygulamanın doğrulama akışını gösterir. Kullanıcı doğru parolayı girmiştir (step_1_status), ancak sistem oturumu açmak yerine doğrulama zincirinin ikinci halkası olan donanımsal güvenlik anahtarından (YubiKey vb.) gelecek kriptografik yanıtı beklemektedir (step_2_status). Bu yapı, zayıf parolaların sömürülmesini (credential exploitation) tamamen etkisiz hale getirir.

Authorisation ve Access Control Mimarisi

Siber güvenlikte **Authentication** (Kimlik Doğrulama) süreci başarıyla tamamlandıktan sonra sistemin karşı karşıya kaldığı bir sonraki kritik aşama, doğrulanmış bu kullanıcının sistem içerisinde hangi sınırlarda hareket edeceğinin belirlenmesi ve dayatılmasıdır. Doğrulanmış bir kimlik, sistemdeki her şeye sınırsız erişim hakkı (root/admin privileges) anlamına gelmez. Bu noktada güvenlik mimarisi iki temel kavrama ayrılır: Kuralı belirleyen **Authorisation** ve kuralı teknik olarak uygulayan **Access Control**.

Authorisation (Yetkilendirme): Kural ve Politikanın Belirlenmesi

Authorisation, kimliği doğrulanmış bir kullanıcının (veya servisin) sistem içerisinde hangi kaynaklara (dosyalar, veri tabanları, ağ segmentleri) erişebileceğini ve bu kaynaklar üzerinde hangi eylemleri (okuma, yazma, silme, yürütme) gerçekleştirmesine izin verildiğini belirleyen mantıksal süreçtir.

Bu süreç, endüstri standardı olan **Principle of Least Privilege - PoLP** (En Az Ayrıcalık Prensibi) üzerine inşa edilir. Bir kullanıcıya sadece işini yapabilmesi için gereken minimum yetkiler verilir.

Analojiler üzerinden konseptin mantığı:

- **Otel Senaryosu:** Otele giriş yaptığınızda (Authentication), sadece kendi kiraladığınız odanın kapısını açmaya yetkili olmanız, diğer misafirlerin odalarına veya personelin kullandığı arka plan odalarına (örneğin elektrik odası) girememeniz bir **Authorisation** kararıdır.
- **Spor Salonu Senaryosu:** Aidatını ödemiş bir üyenin çalışma saatleri içinde tüm ekipmanları kullanabilmesi, ancak koşu bandını söküp bir aylığına evine götürmemesi yetki sınırlarının (Scope of Authorisation) belirlenmesidir.

ÖNEMLİ: Authorisation sadece ne yapabileceğinizi söyleyen bir "izinler bütünü" veya "politikadır". Kendi başına fiziksel veya yazılımsal bir engelleyici değildir. Kararın sahada uygulanması için başka bir mekanizmaya ihtiyaç vardır.

Access Control (Erişim Kontrolü): Politikanın Teknik Olarak Dayatılması

Access Control, **Authorisation** aşamasında belirlenen teorik politikaları ve kuralları, fiziksel veya mantıksal sistemler üzerinde zorla uygulayan (enforce) mekanizmaların bütünüdür. "Kimlik X'in, Y kaynağına erişim yetkisi yok" diyen yapı Authorisation iken; Kimlik X, Y kaynağına erişmeye çalıştığında önüne "Access Denied" (Erişim Reddedildi) hatası çıkartan veya kapıyı kitleyen sistem **Access Control** mekanizmasıdır.

- Otel örneğindeki mantıksal yetkilendirme kararını (sadece kendi odana girebilirsin) sahada uygulayan şey, odanın kapısındaki **elektronik kart okuyucusu ve akıllı kartın (Smart Card) kriptografik eşleşmesidir**.
- Spor salonundaki koşu bandını dışarı çıkarmanızı engelleyen şey, kapıdaki turnikeler, güvenlik kameraları veya donanım üzerindeki fiziksel kilitlerdir.

Kurumsal Sistemlerde Entegrasyon: Role-Based Access Control (RBAC)

Teknik bir ortamda bu iki kavramın nasıl iç içe çalıştığını inceleyelim. Kurumsal bir yapıda Satış (Sales) departmanında çalışan "Edward" isimli bir kullanıcıyı ele alalım. Edward'ın işini verimli bir şekilde yapabilmesi için müşteri veri tabanına ve satış raporlarına erişmesi gerekir. Ancak İnsan Kaynakları (HR) departmanının maaş bordrolarına veya Muhasebe (Accounting) departmanının finansal dosyalarına erişimi olmamalıdır.

Bu senaryoda sistem yöneticisi (SysAdmin) genellikle **RBAC (Role-Based Access Control - Rol Tabanlı Erişim Kontrolü)** mimarisini kullanır. Edward'a tek tek dosya izni vermek yerine, onu "Sales_Team" isimli bir gruba (Role) dahil eder. **Authorisation** politikası, "Sales_Team sadece /shared/sales/ dizinini okuyabilir ve yazabilir" şeklinde belirlenir.

Bu politikanın sistemde **Access Control** olarak nasıl dayatıldığına Linux dosya sistemi üzerinden bakalım:

```
# Kurumsal sunucudaki dizin izinlerinin (Access Control) kontrol edilmesi
ls -ld /shared/sales /shared/hr_payroll
drwxrwx--- 2 root Sales_Team 4096 May 20 10:00 /shared/sales
drwxr-x--- 2 root HR_Team 4096 May 20 10:05 /shared/hr_payroll
```

Yukarıdaki çıktı, **Access Control** mekanizmasının kalbidir. `/shared/hr_payroll` dizini üzerindeki `drwxr-x---` izinleri (Linux File Permissions), bu dizine sadece "root" kullanıcısının ve "HR_Team" grubunun erişebileceğini kesin bir dille sisteme dikte eder. Edward ("Sales_Team" üyesi) doğrulanmış bir kullanıcı (Authenticated) olmasına rağmen, bu dizine girmeye çalıştığında sistemin **Access Control** mekanizması devreye girecek ve izin vermeyecektir.

```
# Edward kullanıcısının yetkisiz bir dizine erişim denemesi
[edward@corp-server ~]$ cd /shared/hr_payroll
bash: cd: /shared/hr_payroll: Permission denied
```

Bu *Permission denied* (Erişim engellendi) hatası, **Authorisation** (yetkin yok politikası) kararının, dosya sistemi **Access Control** mekanizması tarafından sahada uygulanmasının (enforcement) teknik sonucudur.

Uygulama Seviyesinde Erişim Kontrolü: Mail Sunucuları

Kavramı e-posta hesapları üzerinden özetlersek: Kendi e-posta adresinize kullanıcı adı ve parolanızla giriş yapmanız **Authentication**'dir. Sistemin, sizin sadece kendi gelen kutunuzu (Inbox) okuyabileceğinize ve iş arkadaşlarınızın mailerini okuyamayacağınıza karar vermesi **Authorisation**'dir. E-posta sunucusunun (örneğin Exchange veya Postfix) arka planda sizin oturum token'inizi kontrol ederek diğer mailbox veri tabanlarına yapacağınız okuma isteklerini (`GET /mailbox/colleague`) yazılımsal olarak düşürmesi (Drop/Deny) ise **Access Control**'dür.

DİKKAT: Sınavlarda ve teknik mülakatlarda bu iki kavram sıklıkla birbirinin yerine kullanılarak tuzak kurulur. Kuralı aklınızda şu şekilde sabitleyin:

- **Authorisation (1)** → Karar verir, politikayı çizer (What a user *should* be able to access).
- **Access Control (2)** → Harekete geçer, politikayı uygular ve yasaklar (Enforces the set policy).

IAM ve Güvenlik Mimarisinde Accountability (Hesap Verilebilirlik) ve Merkezi Loglama (Logging)

Siber güvenlikte, bir kullanıcının kimliğini doğrulaması (Authentication) ve belirli kaynaklara erişim yetkisi alması (Authorisation) sürecin sadece başlangıcıdır. Sistemin gerçek güvenliği,

bu kullanıcının içerde ne yaptığına ve eylemlerinden nasıl sorumlu tutulacağına bağlıdır.

Accountability (Hesap Verilebilirlik), kullanıcının sistem üzerindeki tüm hareketlerinin inkar edilemez (non-repudiation) bir şekilde kendisine bağlanması sürecidir. Bu mekanizma, kapsamlı bir **Auditing** (Denetleme) kapasitesi gerektirir ve bu kapasitenin yegane yakıtı doğru yapılandırılmış **Logging** (Loglama/Kayıt Tutma) işlevidir.

İç Tehditler (Insider Threats) ve Auditing Gereksinimi

Yetkilendirilmiş kullanıcıların her zaman iyi niyetli veya güvende olduğu varsayılmaz (Zero Trust mimarisinin temel kurallarından biri). Bir banka memuru senaryosunu ele alalım. Bu personel, iş tanımı gereği müşterilerin hesap detaylarını okuma ve işlem yapma yetkisine sahiptir. Ancak, bu memurun (veya hesabı ele geçirilmiş memurun) bu yetkiyi kötüye kullanmasını (rogue employee) engelleyecek mekanizma erişim kontrolü değildir; çünkü zaten yetkisi vardır.

Sistemin güvenilirliğini sağlayan tek şey, yapılan her işlemin, işlemi yapanın kimliği, zamanı ve işlem detayıyla birlikte güvenli bir şekilde kaydedilmesidir. Eğer bir ihlal gerçekleşirse, denetleme (Auditing) mekanizmaları bu logları inceler ve "Kim, ne zaman, nerede, ne yaptı?" sorularına kesin cevaplar verir.

Sistemin Hafızası: Logging (Kayıt Tutma) ve Anomali Tespiti

Logging, bir sistemde meydana gelen kullanıcı eylemlerinin, sistem olaylarının, ağ trafiklerinin ve hataların (errors) kronolojik bir şekilde kaydedilmesi işlemidir. Kusursuz işleyen bir loglama sistemi; mevzuat uyumluluğu (Regulatory Compliance), olay müdahalesi (Incident Response) ve adli bilişim soruşturmaları (Forensic Investigations) için kurumsal yapının en hayati bileşenidir.

Log verilerinin analizi, güvenlik ekiplerine sadece geçmişini değil, o an gerçekleşen şüpheli aktiviteleri de gösterir.

```
# Örnek bir /var/log/auth.log çıktısının analizi
May 22 03:14:15 db-server sshd[15432]: Failed password for root from
192.168.10.50 port 49152 ssh2
May 22 03:14:18 db-server sshd[15435]: Failed password for root from
192.168.10.50 port 49154 ssh2
May 22 03:14:21 db-server sshd[15438]: Accepted password for root from
192.168.10.50 port 49156 ssh2
May 22 03:15:00 db-server sudo[15500]: root : TTY=pts/0 ; PWD=/root ;
USER=root ; COMMAND=/bin/tar -czvf /tmp/db_dump.tar.gz /var/lib/mysql/
```

Yukarıdaki log kaydı, **Accountability** ilkesinin sahada nasıl çalıştığını gösterir. İlk iki satırda, **192.168.10.50** IP adresinden **root** kullanıcısı için başarısız giriş denemeleri görüyoruz. Üçüncü satırda giriş başarılı oluyor. Saat **03:14**'te (muhtemelen mesai saatleri dışında) gerçekleşen bu işlem başlı başına bir anomalidir. Dördüncü satırda ise bu kullanıcının, kritik veritabanı dizinini (`/var/lib/mysql/`) arşivlediğini (`tar` komutu) görüyoruz. Eğer bu

sistemde doğru bir loglama olmasaydı, verilerin sızdırıldığı (Data Exfiltration) anlaşılamayacak ve fail meçhul kalacaktı. Yetkisiz erişim veya anomali anında log sistemi otomatik olarak bir uyarı (Alert) üreterek SOC (Security Operations Center) ekibini harekete geçirmelidir.

Kanıtların Korunması: Tamper-Proofing ve Log Forwarding (Log Yönlendirme)

DİKKAT: Bir sisteme sızan ve ayrıcalıklarını yükselten (Privilege Escalation) bir saldırganın yapacağı ilk iş, izlerini silmek için log dosyalarını (örneğin Linux'ta `/var/log` dizinini veya Windows'ta Event Viewer kayıtlarını) silmek veya değiştirmektir (Log Tampering).

Bu zafiyeti önlemek için logların "değiştirilemez" (Tamper-proof) olması zorunludur. Kurumsal mimarilerde bu, **Log Forwarding** (Log Yönlendirme) ile sağlanır.

Log Forwarding, bir uç noktadaki (sunucu, firewall, router) log verilerinin, o sistemde tutulmak yerine anında ağ üzerinden izole edilmiş, merkezi bir log sunucusuna veya bulut tabanlı bir depolama servisine gönderilmesidir.

```
# Örnek bir /etc/rsyslog.conf Log Forwarding yapılandırması
*. * @192.168.100.10:514
```

Yukarıdaki yapılandırma komutunda (Linux `rsyslog` servisi), `*. *` parametresi "tüm tesislerden (facilities) gelen tüm seviyelerdeki (severities) logları" ifade eder. `@@` parametresi verinin güvenilir TCP protokolü üzerinden gönderileceğini belirtir, **192.168.100.10** merkezi log sunucusunun IP adresi, **514** ise standart Syslog portudur. Bu yapı sayesinde saldırgan hedef sunucuyu tamamen ele geçirse bile, eylemleri anında merkezi sunucuya iletildiği için geçmişini silemez.

Veriyi Zekaya Dönüştürmek: SIEM (Security Information and Event Management)

Yüzlerce sunucudan, ağ cihazından ve uygulamadan saniyede binlerce log satırı gelir. Bu devasa veriyi manuel olarak incelemek imkansızdır. İşte bu noktada **SIEM** devreye girer.

SIEM, ağdaki tüm noktalardan gelen log verilerini merkezi bir konumda toplayan (Aggregation) ve bu verileri siber tehdit belirtilerine karşı analiz eden ileri düzey bir teknolojidir. SIEM'in asıl gücü **Correlation** (İlişkilendirme) yeteneğidir.

Örneğin; bir güvenlik duvarı dışarıdan gelen bir port taramasını (Port Scan) loglar, aynı saniyelerde Active Directory bir kullanıcının hesabında ardışık parola hataları loglar, hemen ardından veritabanı sunucusu o kullanıcının yüklü miktarda veri çektiğini loglar. Tek başlarına incelendiğinde bu olaylar gözden kaçabilir, ancak SIEM bu logları birleştirerek "Sistemde aktif bir veri sızıntısı saldırısı (Data Breach) var" sonucunu çıkarır ve kritik alarm üretir.

SIEM ve Logging Entegrasyonunun Çıktıları:

1. **Incident Response (Olay Müdahalesi):** Potansiyel tehditlerin anında tespit edilip, saldırganın sistemde yatayda yayılmasının (Lateral Movement) engellenmesi.
2. **Forensic Investigations (Adli Bilişim Soruşturmaları):** Bir ihlalden sonra, sorunun kaynağını (Root Cause Analysis), saldırganın giriş vektörünü ve etkilenen sistemleri saat saat haritalandırmak için gerekli detaylı sistem aktivite tarihçesinin sağlanması.
3. **Compliance Reporting (Uyumluluk Raporlaması):** PCI-DSS, GDPR veya HIPAA gibi yasal mevzuatlar gereği kurumların denetim (Audit) süreçlerinde yetkili kurumlara sunması gereken, manipüle edilmemiş erişim verilerinin hazır bulundurulması.

IAM ve IdM Mimarisi: Kimlik ve Erişim Yönetimi Dinamikleri

Siber güvenlik mimarisinde sistemlere kimin eriştiğini ve ne yapmaya yetkili olduğunu yönetmek, güvenliğin en temel yapı taşıdır. Bu yönetim süreci, sektörde sıklıkla birbirinin yerine kullanılan ancak teknik derinlikte farklı sorumluluklara sahip iki temel konsept etrafında şekillenir: **Identity Management - IdM** (Kimlik Yönetimi) ve **Identity and Access Management - IAM** (Kimlik ve Erişim Yönetimi).

Bu iki kavram arasındaki temel farkı anlamak, kurumsal ağların ve bulut altyapılarının güvenlik duruşunu (security posture) doğru yapılandırmak için kritik bir gereksinimdir.

Identity Management (IdM): Sistemin Dijital Kimlik Altyapısı

IdM, bir organizasyonun ağındaki kullanıcı kimliklerinin yönetilmesi, doğrulanması (Authentication) ve yetkilendirilmesi (Authorisation) süreçlerine odaklanan temel siber güvenlik bileşenidir. Temel amacı, her bir kullanıcıya, cihaza veya servise sistem içerisinde benzersiz bir **dijital kimlik** atamaktır.

IdM sistemleri, kullanıcı kimliklerini ve temel erişim haklarını depolamak için merkezi bir veritabanı (örneğin Active Directory, LDAP veya FreeIPA) kullanır. Bu merkezi yapı, yetkisiz erişim riskini azaltırken yönetimi basitleştirir.

IdM'in merkezinde yer alan en kritik teknik operasyon **User Provisioning** (Kullanıcı Hazırlama/Tahsis Etme) sürecidir. Provisioning, bir kullanıcının sisteme dahil olduğu anda dijital kimliğinin oluşturulması ve departmanına/rolüne uygun temel niteliklerin (attributes) bu kimliğe tanımlanmasıdır.

```
# Merkezi bir IdM veritabanında (LDAP) bir kullanıcının dijital kimliğinin (User Provisioning) sorgulanması
ldapsearch -x -H ldap://192.168.1.10 -b "dc=corp,dc=local" "(sAMAccountName=jdoe)"
```

Yukarıdaki `ldapsearch` komutu, IdM altyapısının sahadaki yansımasıdır. `-H` parametresi ile hedef IdM sunucusu belirtilir, `-b` (base) parametresi ile arama yapılacak dizin tabanı gösterilir ve `sAMAccountName=jdoe` filtresi ile "jdoe" isimli kullanıcının dijital kimliği ve

nitelikleri (attributes) merkezi veritabanından çekilir. IdM'in görevi bu kaydı oluşturmak, korumak ve kimlik doğrulama isteklerine yanıt vermektir.

Identity and Access Management (IAM): Kapsamlı Erişim ve Yönetişim Çerçevesi

Eğer IdM kimliğin kendisini oluşturuyorsa, **IAM** bu kimliğin şirket politikalarına, yasal regülasyonlara ve anlık durumlara göre ne yapabileceğini değerlendiren ve dayatan (enforce) çok daha kapsamlı bir üst şemsiyedir.

IAM, sadece kimlikleri yönetmekle kalmaz; erişim kontrolünü, kimlik yönetişimini (Identity Governance) ve uyumluluk yönetimini (Compliance Management) de kapsar. IAM sistemleri, IdM tarafından sağlanan nitelikleri (attributes) değerlendirir ve erişim kararını dinamik olarak verir.

Bu kararları verirken IAM sistemleri aşağıdaki kritik teknolojileri tek bir merkezde entegre eder:

- **Role-Based Access Control - RBAC (Rol Tabanlı Erişim Kontrolü):** Kullanıcılara tek izin vermek yerine, onları organizasyonel rollerine göre gruplayarak erişim yetkilerini dinamik olarak atama mantığıdır.
- **Multi-Factor Authentication - MFA (Çok Faktörlü Kimlik Doğrulama):** IdM'in temel kimlik doğrulamasını alıp, üstüne SMS, donanımsal token veya biyometri gibi ek doğrulama katmanları ekleyen sistemdir.
- **Single Sign-On - SSO (Tek Oturum Açma):** Kullanıcının bir kez kimlik doğrulamasından geçtikten sonra, arka planda token (örneğin SAML veya OAuth) alışverişi ile tüm yetkili servislere tekrar parola girmeden erişebilmesini sağlayan mimaridir.

ÖNEMLİ: IAM mimarisinin en kritik görevlerinden biri **Kimlik Yaşam Döngüsü Yönetimidir** (Identity Lifecycle Management). Bir kullanıcının işe başlamasıyla (Onboarding) kimliğinin aktif edilmesi, işten ayrılmasıyla (Offboarding) veya rol değiştirmesiyle birlikte tüm erişimlerinin anında geri alınması (Access Revocation) IAM'in sorumluluğundadır. Bu döngüdeki bir zafiyet, sektördeki yetki yükseltme (Privilege Escalation) saldırılarının bir numaralı sebebidir.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::finance-data",
      "Condition": {
        "Bool": {
          "aws:MultiFactorAuthPresent": "true"
        }
      }
    }
  ]
}
```

```
}  
  }  
    }  
  ]  
}
```

Yukarıdaki JSON çıktısı, bir AWS IAM Policy (Erişim Politikası) örneğidir ve IAM'in IdM'den nasıl ayrıldığı kusursuzca özetler. Kullanıcının kimliği IdM tarafından doğrulanmış olsa bile, IAM politikası devreye girerek şunu söyler: "Erişime İzin Ver (*Effect: Allow*), ancak sadece finans verilerini listeleyebilir (*Action: s3:ListBucket*) ve bu işlemi YALNIZCA MFA (Çok Faktörlü Doğrulama) ile giriş yaptıysa gerçekleştirebilir (*Condition: MultiFactorAuthPresent: true*)." IAM'in asıl gücü bu koşullu erişim mimarisidir.

Regülasyon Uyumluluğu ve Denetim (Compliance & Auditing)

DİKKAT: Modern kurumsal yapılarda IAM sistemleri sadece güvenlik için değil, yasal zorunluluklar için de kurulur. Sağlık verilerini koruyan **HIPAA**, Avrupa Birliği veri gizliliği standartlarını belirleyen **GDPR** veya kredi kartı ödeme sistemleri güvenliğini sağlayan **PCI DSS** gibi regülasyonlar, sistemlere kimin, ne zaman ve hangi veriye eriştiğinin saniye saniye loglanmasını (Auditing) şart koşar.

İyi yapılandırılmış bir IAM sistemi, kullanıcı aktivitelerini takip ederek bu denetim raporlarını otomatik olarak oluşturur. İhtimal dahilindeki bir veri sızıntısında (Data Breach), olayın kaynağının tespit edilmesi (Forensic Investigation) tamamen IAM sisteminin tuttuğu bu hesap verilebilirlik (Accountability) kayıtlarına dayanır.

Özetle IdM ve IAM Arasındaki İnce Çizgi

Bazı kaynaklar bu iki terimi eşanlamlı kullansa da, teknik mimaride ayrım şudur:

- **IdM (Kimlik Yönetimi):** "Bu kullanıcı kimdir ve dijital dünyadaki nitelikleri nelerdir?" sorusuna cevap verir. Merkezi bir kimlik veritabanı kurmaya odaklanır.
- **IAM (Kimlik ve Erişim Yönetimi):** "Bu doğrulanmış kullanıcının şirket politikaları çerçevesinde şu an, bu cihazdan, bu kaynağa erişmesine izin vermeli miyim?" sorusuna cevap verir. Politikaları, denetimleri, RBAC'i ve yaşam döngüsünü yöneten çatı sistemdir.

Kimlik Doğrulama Mekanizmalarına Yönelik Saldırılar (Attacks Against Authentication)

Siber güvenlik mimarisinde kimlik doğrulama (Authentication), sistemlerin kapısındaki en kritik savunma hattıdır. Bu alan, amatörce tasarlanmış, "kendi kriptografini kendin yaz" (roll-your-own-crypto) mantığıyla oluşturulmuş naif protokollere kesinlikle tahammül etmez. Sektörel bir kural olarak; endüstri standartlarında akran denetiminden (peer-reviewed)

geçmiş, test edilmiş yapıların kullanılmaması, sistemin er ya da geç aşılmasıyla sonuçlanacaktır.

Aşağıdaki analizde, teorik bir kimlik doğrulama protokolünün zafiyetlerini ve bu zafiyetlerin mantıksal evrimini inceleyeceğiz.

Fiziksel Dünyadaki Zafiyetler: Eavesdropping (Gizli Dinleme)

Konsepti anlamak için fiziksel bir benzetmeyle başlayalım: Sadece üyelerin alındığı özel bir toplantıda, kapıdaki görevli üyeleri tanımadığı için bir parola ("Yedi At") sorarak kimlik doğrulaması yapıyor. Bu sistem, dışarıdan biri kapıya yeterince yaklaşıp parolayı duyana kadar kusursuz çalışır.

Fiziksel dünyada bir saldırganın parolayı ele geçirmesine **Eavesdropping** (Kulak misafiri olma/Gizli dinleme) denir. Parolayı çoklamak (10 farklı soru-cevap belirlemek) geçici bir çözümdür, zira dinleme yeteneğine sahip saldırgan yeterli sürede tüm kombinasyonları öğrenecektir. Fiziksel dünyada güvenlik, etrafta gezinen şüpheli kişilerin gözlemlenmesiyle sağlanabilirken; dijital ağlarda saldırgan tamamen görünmezdir.

Dijital Dünyada İlk Hata: Cleartext (Düz Metin) Kimlik Doğrulama

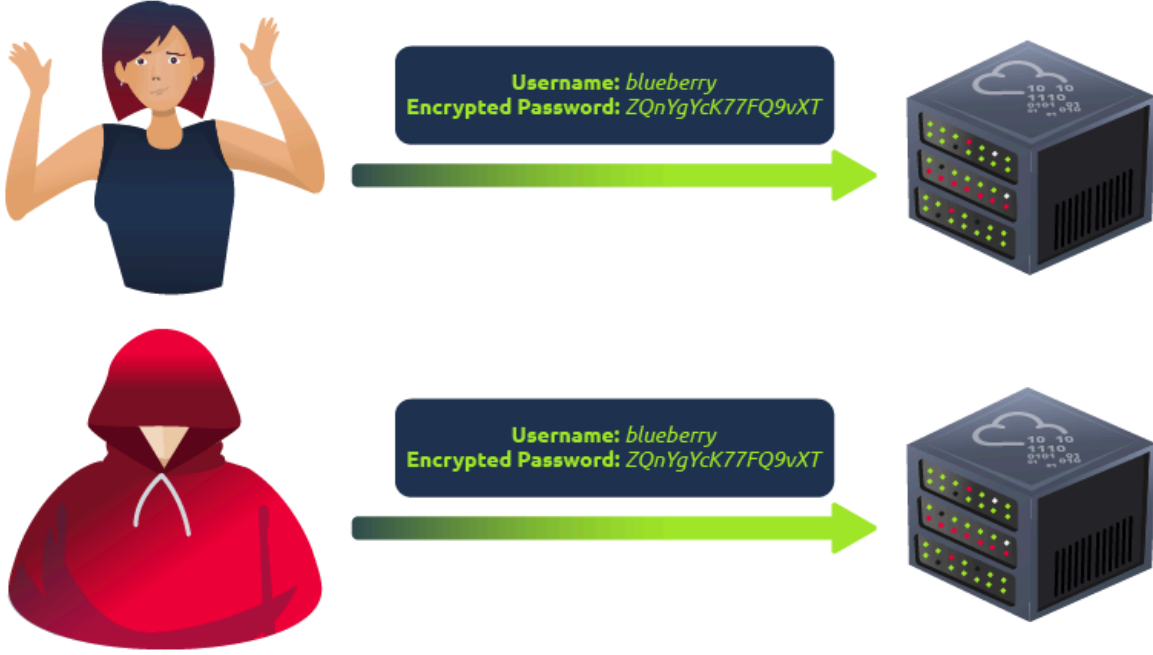
Ağ üzerindeki kimlik doğrulama süreçleri, fiziksel dünyadan çok daha acımasızdır. Eğer bir sistem kullanıcı adı ve parolasını **Cleartext** (şifrelenmemiş, düz metin) olarak iletiyorsa, ağ trafiğini dinleyen (Packet Sniffing) herhangi bir saldırgan, Wireshark veya tcpdump gibi araçlarla kimlik bilgilerini anında ele geçirebilir.

```
POST /api/v1/login HTTP/1.1
Host: secure-corp.local
Content-Type: application/x-www-form-urlencoded

username=admin&password=SuperSecretPassword123
```

Yukarıdaki örnek HTTP isteğinde kimlik bilgileri düz metin olarak iletilmektedir. Ağda konumlanmış bir saldırgan (örneğin ARP Spoofing ile araya girmiş biri), bu paketi yakaladığında hiçbir şifre kırma işlemine (Cracking) gerek duymadan doğrudan sisteme

eriřim saęlar.



Savunma Giriřimi ve lmcl Zafiyet: Replay Attack (Tekrar Oynatma Saldırısı)

Dz metin iletimi zafiyetini kapatmak iin geliřtiriciler genellikle istemci (Client) ve sunucu (Server) arasında sabit bir gizli anahtar (Fixed Secret Key) belirler. Kullanıcı parolası bu anahtar ile řifrelenir (Encryption) ve aęa yle gnderilir.

Artık aę trafięini dinleyen saldırgan, parolayı dz metin olarak gremez. Ancak bu durum, sistemin gvenli olduęu anlamına gelmez. nk saldırganın **parolanın ne olduęunu bilmesine gerek yoktur**.

Sistem sadece řifrelenmiř verinin (payload) doęruluęunu kontrol ettięi iin, saldırgan yakaladıęı řifrelenmiř paketin tamamını alır ve sunucuya kendi bilgisayarından aynen tekrar gnderir. Sunucu, gelen geerli řifreli metni grdęnde saldırganı meřru kullanıcı sanarak ieri alır. Bu yıkıcı saldırı vektrne **Replay Attack** (Tekrar Oynatma Saldırısı) denir.

```
# Saldırganın aęda yakaladıęı řifrelenmiř meřru kimlik doęrulama paketi
POST /api/v1/secure-login HTTP/1.1
Host: secure-corp.local
Auth-Token: 8f4c2b9a7d1e5f8a002b4c... (řifrelenmiř sabit payload)

# Saldırganın curl komutu ile aynı payload'u sunucuya tekrar gndermesi
(Replay Attack)
curl -X POST http://secure-corp.local/api/v1/secure-login \
-H "Auth-Token: 8f4c2b9a7d1e5f8a002b4c..."
```

Bu noktada arka plandaki mantık hatası řudur: Sunucu, paketin orijinal kullanıcıdan o an mı geldięini, yoksa aęda yakalanmiř eski bir paketin yeniden mi sunulduęunu ayırt

edememektedir. Şifreleme algoritması ne kadar güçlü olursa olsun, kriptografik mimari "tekrar oynatılabilir" (replayable) olduğu için sistem tamamen ele geçirilmiştir.

Kriptografik Çözüm: Unique Challenge-Response (Benzersiz Meydan Okuma-Yanıt)

Replay Attack vektörünü tamamen ortadan kaldırmanın tek yolu, gönderilen her kimlik doğrulama talebinin (request) matematiksel olarak tamamen benzersiz ve yalnızca o an için geçerli olmasını sağlamaktır. Sabit şifreli metinlerin yerini dinamik metinler almalıdır.

Bu mekanizmayı kurmanın en etkili yolu, şifreleme algoritmasının içerisine o anın tarih ve saatini (Timestamp) dahil etmektir.

Nasıl Çalışır?

1. Kullanıcı sisteme giriş yapmak istediğinde, arka planda parola ve o anın UNIX zaman damgası (örn: 1700483921) birleştirilir.
2. Bu iki veri birlikte şifrelenir ve sunucuya gönderilir.
3. Sunucu paketi çözer, parolayı doğrular ve zaman damgasının **geçerlilik penceresi** (genellikle sadece birkaç milisaniye) içinde olup olmadığına bakar.

```
// Modern bir Challenge-Response mimarisinde oluşturulan örnek bir JSON Payload
{
  "username": "admin",
  "timestamp": 1700483921500,
  "secure_hash": "a4b... (SHA256(Password + Timestamp + Salt))"
}
```

Eğer bir saldırgan bu paketi yakalayıp sadece 1 saniye sonra bile sunucuya (Replay) gönderirse, sunucu paketi reddedecektir. Çünkü paketin içindeki zaman damgası sunucunun zamanına göre "bayatlamış" (expired) olacaktır. Saldırgan zaman damgasını manuel olarak güncellerse, `secure_hash` değeri parolayı bilmediği için bozulacak (Integrity Failure) ve paket yine reddedilecektir.

ÖNEMLİ: Bu yapının (Timestamp tabanlı Challenge-Response) başarılı olabilmesi için çok kritik bir sistemsel gereksinim vardır: **Clock Synchronization (Saat Senkronizasyonu)**. Hem istemcinin hem de sunucunun saatlerinin milisaniye bazında eşleşmesi gerekir. Bu nedenle güvenli kimlik doğrulama sistemleri, ağdaki cihazların saatlerini eşitleyen **NTP (Network Time Protocol)** altyapısına doğrudan bağımlıdır. Saatlerin senkronize olmadığı durumlarda meşru kullanıcılar da sisteme giriş yapamaz (Denial of Service durumu oluşur).

Erişim Kontrol Modelleri (Access Control Models)

Bir bilgi sisteminde, kimliği doğrulanmış (Authenticated) bir kullanıcının hangi kaynaklara erişebileceğine karar verme sürecine **Authorisation** (Yetkilendirme) denir. Ancak bu

yetkilendirme kararlarının sistem üzerinde teknik olarak nasıl dayatılacağı (enforcement), mimari seviyede seçilen **Access Control** (Erişim Kontrolü) modeline bağlıdır.

Siber güvenlikte kaynakları, verileri ve süreçleri korumak için kullanılan temel erişim kontrol modelleri, yönetimin kimde olduğuna ve kuralların nasıl işletildiğine göre üç ana kategoriye ayrılır: **DAC**, **RBAC** ve **MAC**.

Discretionary Access Control (DAC) - İsteğe Bağlı Erişim Kontrolü

Discretionary Access Control (DAC), erişim kararlarının tamamen kaynağın sahibine (Resource Owner) bırakıldığı, en esnek ancak yönetimi en zor olan erişim kontrol modelidir. "Discretionary" (isteğe bağlı/takdire bağlı) kelimesi, veriyi oluşturan veya verinin sahibi olan kullanıcının, bu veri üzerindeki hakları diğer kullanıcılara kendi inisiyatifiyle dağıtabilmesini ifade eder.

Günlük hayatta bulut depolama sistemlerinde (örneğin Google Drive) bir dosyayı **특정** (spesifik) bir meslektaşınızla veya ailenizle paylaşmanız, DAC modelinin en net uygulamasıdır. Linux veya Windows dosya sistemlerindeki standart izin yapıları da temelinde DAC mimarisini kullanır.

```
# Linux sistemlerde DAC modelinin saha uygulaması
ls -l /home/alice/project_secret.txt
-rw-r----- 1 alice team_alpha 1024 May 24 14:00 project_secret.txt
```

*Yukarıdaki standart Linux izin listeleme çıktısı, DAC mimarisinin kalbidir. Dosyanın sahibi **alice** kullanıcısıdır. Alice, kendi inisiyatifiyle (DAC) dosya üzerinde kendisine okuma/yazma (rw-) yetkisi tanımlarken, **team_alpha** grubuna sadece okuma (r--) yetkisi vermiştir. Diğer tüm kullanıcıların erişimini ise (---) kısıtlamıştır. Karar mercii tamamen Alice'tir.*

ÖNEMLİ: DAC modeli, küçük ölçekli sistemlerde (aile içi paylaşımlar veya küçük ofisler) kusursuz çalışır. Ancak kurumsal yapılarda, binlerce çalışanın olduğu bir ortamda her kullanıcının yetkilerinin dosya dosya, klasör klasör yönetilmeye çalışılması **Permission Bloat** (İzin Şişmesi) dediğimiz kaotik bir duruma yol açar. Bir çalışanın departmanı değiştiğinde (Cross-boarding), eski dosyalarındaki izinlerini tek tek bulup silmek imkansızdır. Bu ölçeklenebilirlik (Scalability) sorunu, bizi RBAC modeline götürür.

Role-Based Access Control (RBAC) - Rol Tabanlı Erişim Kontrolü

Büyük kurumsal ağlarda (Enterprise Networks) endüstri standardı haline gelmiş model **Role-Based Access Control (RBAC)** mimarisidir. Bu modelde, DAC'ın aksine kullanıcılara doğrudan dosya veya kaynak izni atanmaz. Bunun yerine, organizasyonun yapısına uygun mantıksal **Roller (Roles)** veya gruplar oluşturulur. İzinler kullanıcılara değil, bu rollere atanır. Kullanıcılar ise görevlerine göre bu rollere dahil edilir.

Örneğin, bir muhasebe personelinin finansal tablolara erişmesi gerekirken, Ar-Ge (Araştırma ve Geliştirme) departmanının kaynak kodlarına erişmesi gerekmez. Kullanıcıları

"Muhasebe_Ekibi" ve "ArGe_Ekibi" gibi gruplara ayırdığımızda, yetkilendirme doğrudan bu gruplar üzerinden yapılır.

```
# Bir Linux sisteminde RBAC mantığının gruba kullanıcı eklenerek işletilmesi
usermod -aG ArGe_Ekibi john_doe
```

Yukarıdaki komut, **john_doe** isimli kullanıcıyı **usermod** (user modify) aracı ile **ArGe_Ekibi** grubuna ekler (**-aG** : append to Group). **John**, bu gruba girdiği saniye itibarıyla, sistemdeki **Ar-Ge** klasörlerine atanmış tüm yetkileri otomatik olarak devralır. Gruptan çıkarıldığı an tüm yetkileri düşer. Klasör izinlerine tek tek dokunulmasına gerek kalmaz.

DİKKAT: RBAC mimarisinin en büyük avantajı bakım (maintenance) kolaylığı ve operasyonel verimliliğidir. Yeni bir personel işe başladığında (Onboarding) sadece ilgili role atanır; personel işten ayrıldığında (Offboarding) rolden çıkartılır. Sistem yöneticilerinin dosya bazlı izinlerle boğuşmasını engeller.

Mandatory Access Control (MAC) - Zorunlu Erişim Kontrolü

Mandatory Access Control (MAC), güvenliğin en üst düzeyde (Militarized/Classified Data) tutulduğu, son kullanıcının veya dosya sahibinin hiçbir inisiyatifinin olmadığı en katı erişim kontrol modelidir.

MAC mimarisinde kararlar sistem çekirdeği (Kernel) veya merkezi güvenlik politikaları tarafından zorunlu olarak dayatılır. Bir dosyanın sahibi siz olsanız bile, sistem politikası o dosyayı "Çok Gizli" (Top Secret) olarak etiketlediye ve sizin profiliniz "Sadece Gizli" (Secret) erişimine sahipse, kendi dosyanızı dahi okuyamaz, yetkilerini değiştiremez veya sisteme keyfi yazılımlar (yeni yazılım kurulumu) yükleyemezsiniz.

Linux sistemlerinde bu katı mimariyi uygulamak için işletim sisteminin çekirdeğine entegre edilen iki büyük proje vardır:

1. **AppArmor (Application Armor):** İşletim sistemindeki programların hangi dosyalara erişebileceğini ve hangi eylemleri yapabileceğini profiller (profiles) üzerinden kısıtlayan MAC uygulamasıdır. Özellikle **Debian** ve **Ubuntu** gibi dağıtımlarda standart (default) olarak gelir.
2. **SELinux (Security-Enhanced Linux):** Amerikan Ulusal Güvenlik Dairesi (NSA) tarafından geliştirilmiş, dosya ve süreçleri etiketleme (labeling) mantığıyla çalışan çok daha granüler (ince ayarlı) ve karmaşık bir MAC mimarisidir. **Red Hat**, **CentOS** ve **Fedora** gibi enterprise seviye Linux dağıtımlarında standarttır.

```
# SELinux aktif bir sistemde MAC etiketlerinin görüntülenmesi
ls -Z /etc/shadow
system_u:object_r:shadow_t:s0 /etc/shadow
```

Bu komut (`ls -Z`), sıradan DAC izinlerinin (okuma/yazma) ötesine geçerek dosyanın MAC güvenlik bağlamını (Security Context) gösterir. Burada `shadow_t` etiketi kritik bir rol oynar. Sisteme sızan bir saldırgan `root` yetkilerini ele geçirse (Privilege Escalation yapsa) bile, SELinux politikası "Sadece belirli şifre yönetim araçları `shadow_t` etiketli dosyaları okuyabilir" diyorsa, `root` kullanıcısı `cat /etc/shadow` komutunu çalıştırdığında **Erişim Engellendi (Permission Denied)** hatası alacaktır. MAC, "root" hesabının mutlak gücünü bile sınırlandıran nihai savunma hattıdır.

IAM Mimarisi: Single Sign-On (SSO) ve Merkezi Kimlik Doğrulama

Kurumsal ağlarda kullanıcıların günlük operasyonlarını (iş akışlarını) yürütebilmeleri için e-posta sunucularına, ortak dosya paylaşımlarına (shared files), yazıcılara ve çeşitli iç ağ uygulamalarına erişimleri gerekir. Bu kaynakların her biri, geleneksel sistemlerde kullanıcının kimliğini ispatlaması için ayrı bir **Authentication** (Kimlik Doğrulama) süreci talep eder.

Geleneksel Parola Yönetiminin Zafiyetleri (Password Fatigue)

Parçalı kimlik doğrulama mimarisinde, her sistem kendi kullanıcı veri tabanını tutar. Bir çalışanın bilgisayarını açmak için ayrı, e-postasını kontrol etmek için ayrı, İnsan Kaynakları portalına girmek için ayrı bir parola kullanması beklenir.

Bu durum siber güvenlikte "Parola Yorgunluğu" (Password Fatigue) olarak adlandırılan kritik bir insan faktörü zafiyetine yol açar. Kullanıcılar, haklı olarak onlarca farklı ve karmaşık parolayı akıllarında tutamayacakları için, genellikle aynı parolayı birden fazla sistemde tekrar kullanma (Password Reuse) eğilimi gösterirler.

ÖNEMLİ: Parola tekrarı, bir ağdaki en zayıf halkanın tüm ağı tehlikeye atması anlamına gelir. Saldırgan, güvenliği düşük olan bir iç uygulamayı ele geçirip oradaki parolayı öğrendiğinde, bu parolayı kritik sistemlerde (örneğin VPN veya e-posta) deneyerek (Credential Stuffing) yatayda yayılım (Lateral Movement) gerçekleştirebilir.

Single Sign-On (SSO) Mimarisinin Çözüm Yaklaşımı

Bu operasyonel ve güvenlik kaynaklı problemi çözmek için **Single Sign-On - SSO** (Tek Oturum Açma) mimarisi kullanılır.

SSO, organizasyonların parçalı kimlik doğrulama süreçlerini merkezi bir otoriteye (Identity Provider) bağladığı yapıdır. Kullanıcı, sabah mesaiye başladığında sadece tek bir kimlik bilgisi seti (kullanıcı adı ve parola) ile ana sisteme **Authentication** gerçekleştirir. Sistem bu başarılı girişin ardından kullanıcıya güvenilir bir bilet veya token tahsis eder. Kullanıcı gün içinde diğer servislere gitmek istediğinde, bu servisler kullanıcıya parola sormaz; arka planda SSO altyapısındaki bu güvenilir bileti doğrulayarak erişim (Authorisation) izni verir.

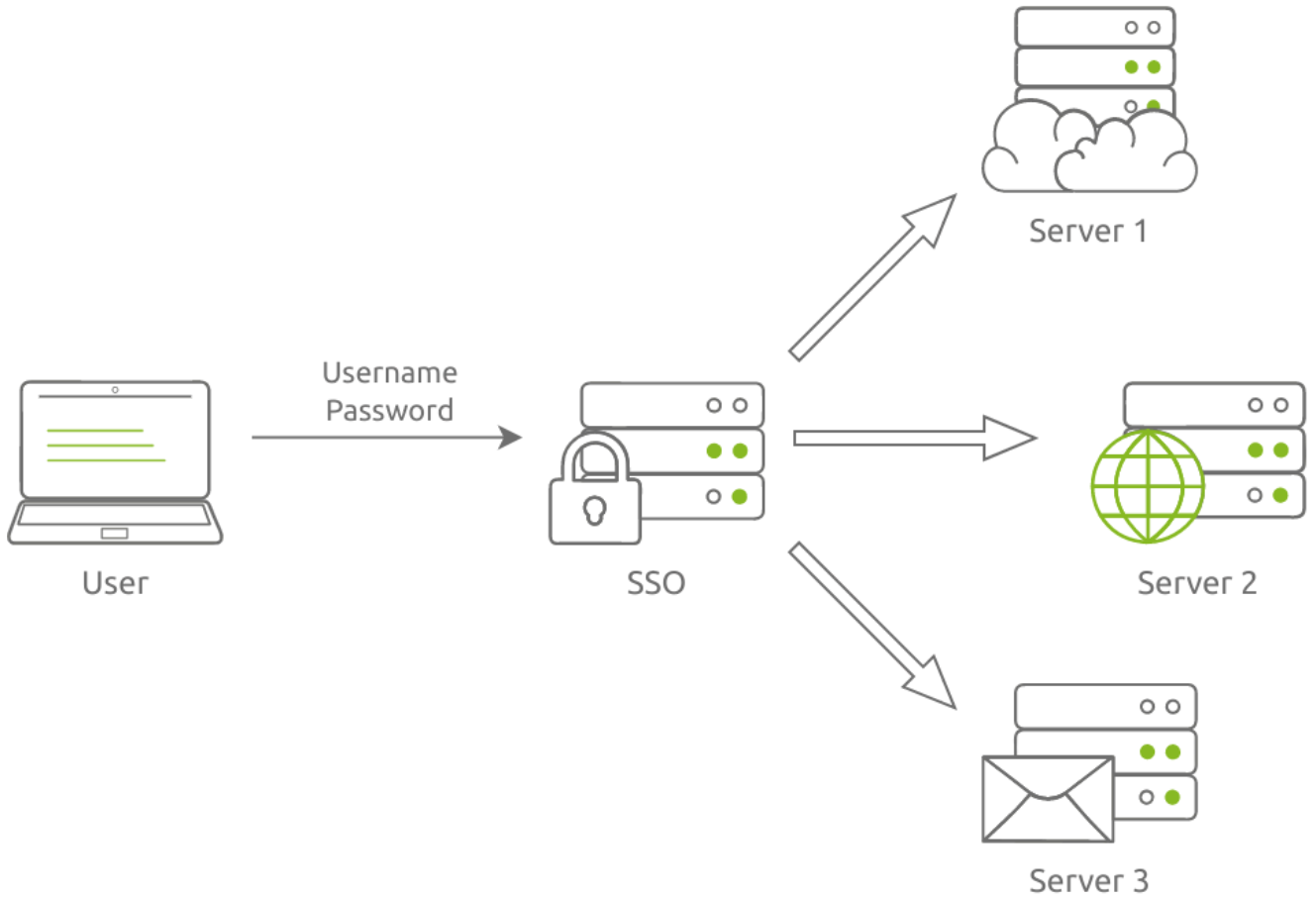
Geleneksel Model vs. SSO Modeli Mantıksal Akışı:

[GELENEKSEL] Kullanıcı -> (Parola 1) -> E-posta Sunucusu
[GELENEKSEL] Kullanıcı -> (Parola 2) -> Dosya Sunucusu
[GELENEKSEL] Kullanıcı -> (Parola 3) -> CRM Uygulaması

[SSO MODELİ]

Kullanıcı -> (Tek Parola) -> Merkezi SSO Sunucusu (Kimlik Doğrulandı, Token Üretildi)
Kullanıcı -> (Token ile) -> E-posta Sunucusu (Erişim Başarılı)
Kullanıcı -> (Token ile) -> Dosya Sunucusu (Erişim Başarılı)
Kullanıcı -> (Token ile) -> CRM Uygulaması (Erişim Başarılı)

Yukarıdaki şema, SSO'nun arka plandaki çalışma mantığını özetler. Kullanıcı parolası sadece merkezi sunucuya iletilir, uç noktalardaki servisler parolayı asla görmez, sadece doğrulanmış oturum bilgisini (token) kabul ederler.



SSO Sisteminin Organizasyonlara Sağladığı Stratejik Avantajlar

SSO mimarisi sadece kullanıcı deneyimini iyileştirmekle kalmaz, aynı zamanda kurumsal güvenlik duruşunu (Security Posture) radikal bir şekilde güçlendirir. Bu avantajlar dört ana teknik başlıkta incelenir:

1. Tek ve Yüksek Entropili Parola (One Strong Password): Kullanıcıdan 10 farklı sistem için 10 ayrı karmaşık parola oluşturmalarını ve bunları düzenli periyotlarla değiştirmesini beklemek gerçekçi değildir. Ancak sistem SSO mimarisine geçirildiğinde, kullanıcıdan

sadece bir adet parola istenir. Bu sayede parola politikaları (Password Policies) çok daha katı hale getirilebilir (örneğin en az 16 karakterli Passphrase kullanımı zorunlu kılınabilir). Kullanıcı sadece tek bir parolayı ezberleyeceği için bu zorluğu kolayca tolere eder.

2. Merkezi ve Basitleştirilmiş MFA (Easier MFA): MFA (Multi-Factor Authentication - Çok Faktörlü Kimlik Doğrulama), sistemlere sadece parola ile girilmesini engelleyen hayati bir güvenlik katmanıdır. Geleneksel yapıda, şirketteki 20 farklı servise ayrı ayrı MFA entegre etmek ve bunları yönetmek, IT ekipleri için sürdürülemez bir yüküdür (Humongous task). SSO sayesinde, MFA sadece merkezi giriş noktasına (Gateway) bir kez kurulur ve yapılandırılır. Kullanıcı sabah SSO'ya girerken telefonuna gelen kodu veya donanımsal anahtarını kullanır; arkasındaki tüm sistemler bu sayede MFA ile korunmuş olur.

3. Operasyonel Destek Kolaylığı (Simpler Support): Geleneksel ağlarda Helpdesk (Destek) ekiplerinin zamanının büyük bir çoğunluğu "Parolamı unuttum" veya "Hesabım kilitlendi" talepleriyle (Password Reset) geçer. Yüzlerce kullanıcının onlarca farklı servisteki parolalarını sıfırlamak devasa bir zaman kaybıdır. SSO mimarisinde tüm kimlikler tek bir hesap altında birleştiği için (Single point of reference), parola sıfırlama veya hesap kilit açma işlemleri son derece basit, hızlı ve tek merkezden yönetilebilir hale gelir.

4. Sistem Verimliliği (Efficiency): Kullanıcıların gün içinde farklı sekme ve uygulamalara her geçişinde yeniden giriş ekranıyla (Login Prompt) karşılaşması, bağlam geçişi (Context Switching) yaratarak verimliliği düşürür. SSO, bu sürtünmeyi ortadan kaldırır. Kullanıcı bir kez doğrulanır ve ihtiyaç duyduğu her yeni servise arka planda pürüzsüz bir şekilde erişir.

DİKKAT: SSO her ne kadar güvenliği ve hızı artırsa da, doğası gereği "Tek Nokta Zafiyeti" (Single Point of Failure) yaratır. Eğer bir saldırgan kullanıcının ana SSO parolasını (veya MFA oturumunu) ele geçirirse, tek bir saldırıyla kullanıcının yetkili olduğu *tüm* kurumsal kaynaklara anında erişim sağlar. Bu nedenle SSO sistemlerinin giriş kapısı her zaman Sıfır Güven (Zero Trust) politikaları, lokasyon bazlı erişim kontrolleri ve sıkı bir MFA altyapısı ile korunmalıdır.